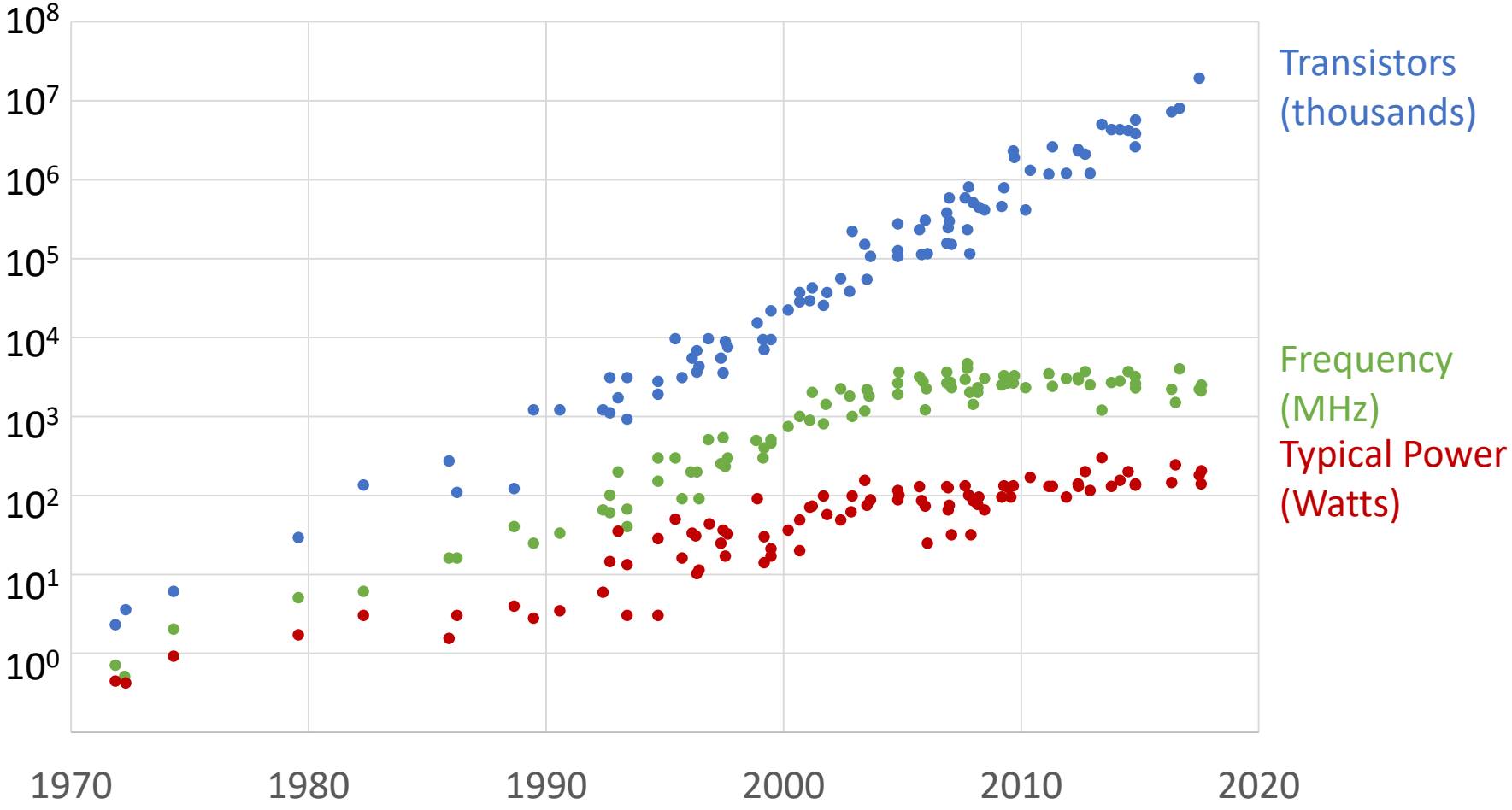
Lecture 35: Parallel Computing

CMPS 221 – Computer Organization and Design

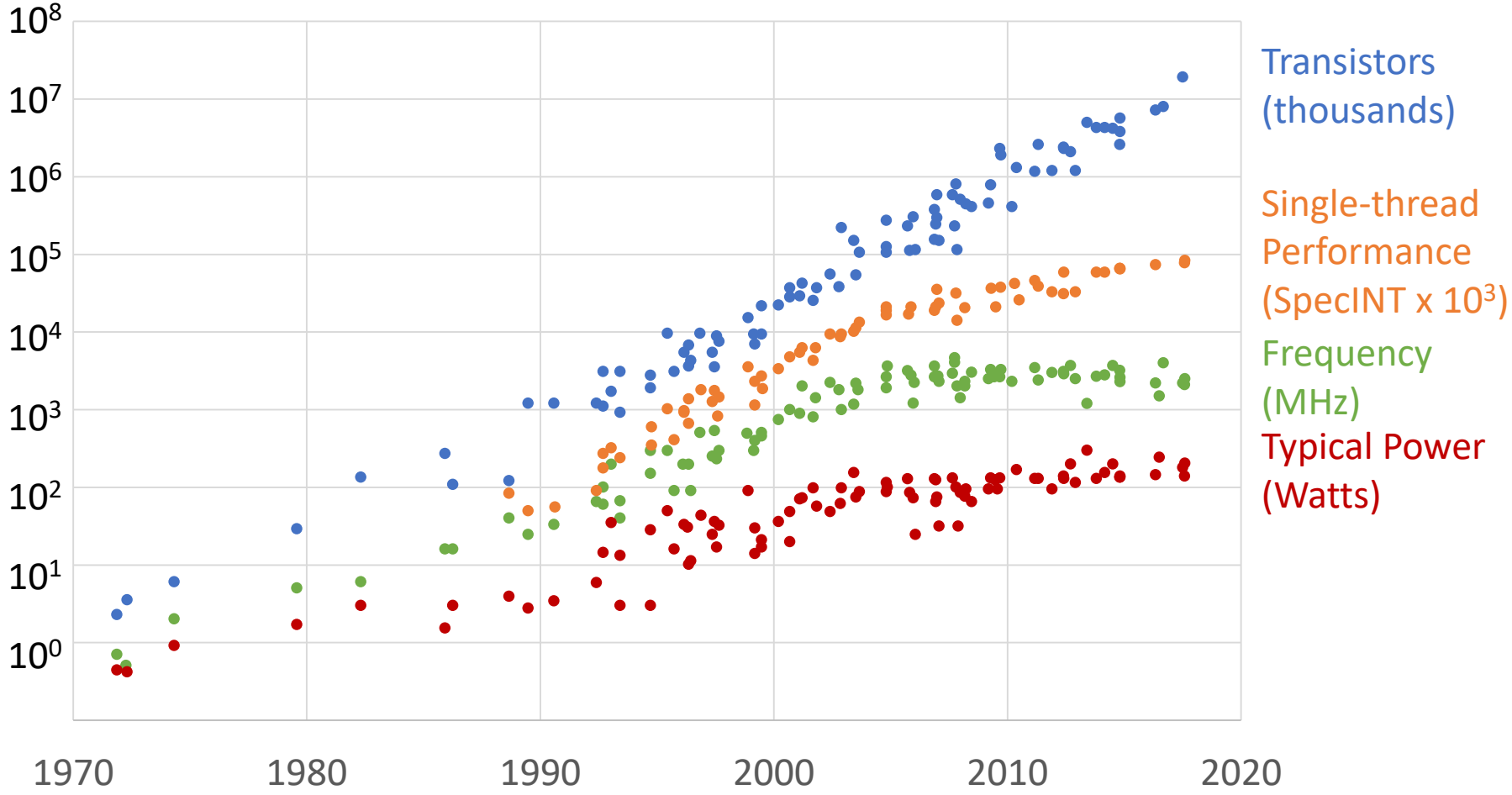
Adapted by Izzat El Hajj from textbook slides for Computer Organization and Design, by David Patterson and John Hennessy, Morgan Kaufmann.

Recall: Processor Trends



Source: M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, C. Batten (1970-2010). K. Rupp (2010-2017).

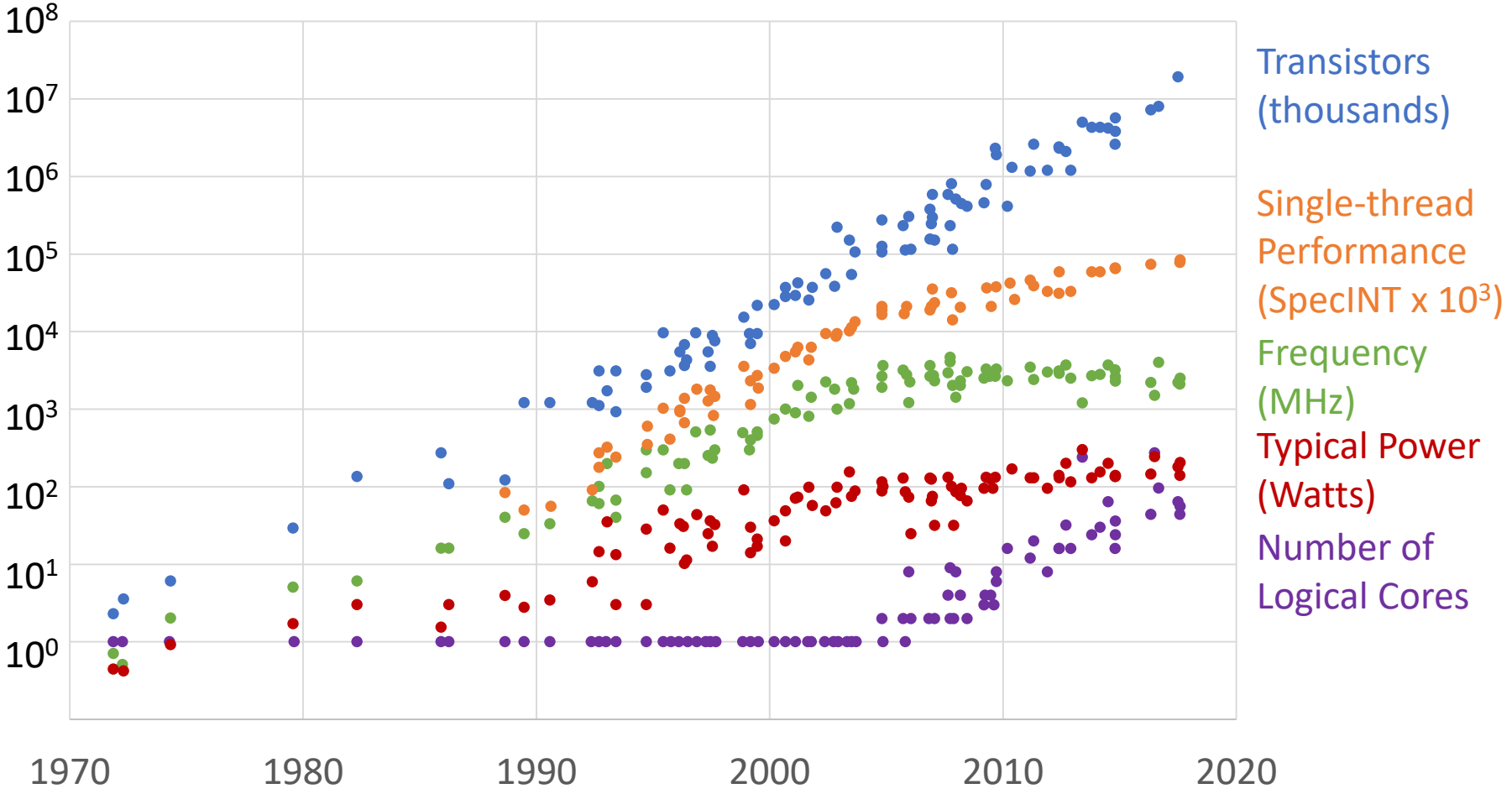
Processor Trends



Source: M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, C. Batten (1970-2010). K. Rupp (2010-2017).

Stagnation in frequency led to a stagnation in **single-thread performance**, except for some improvements due to architecture and compiler advancements

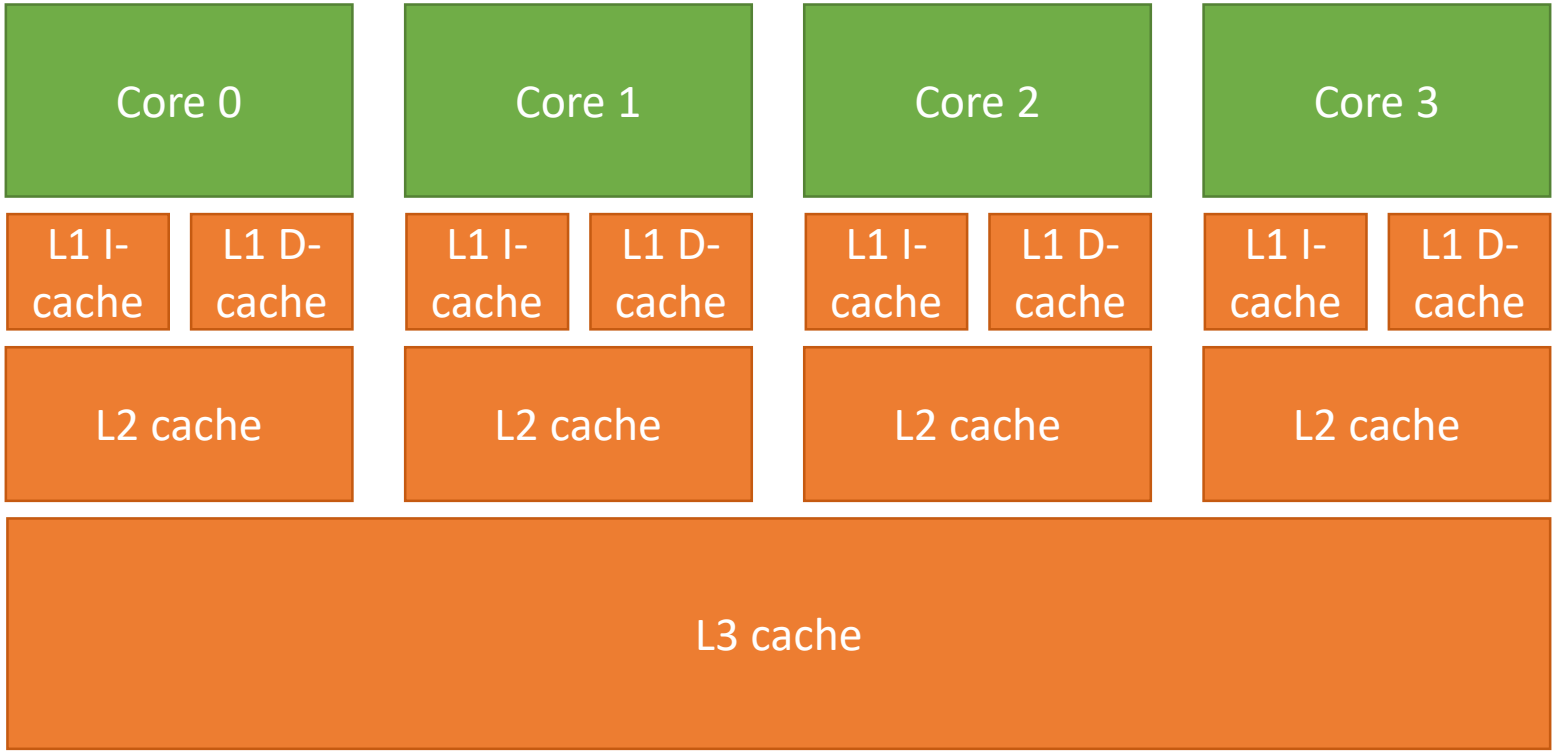
Processor Trends



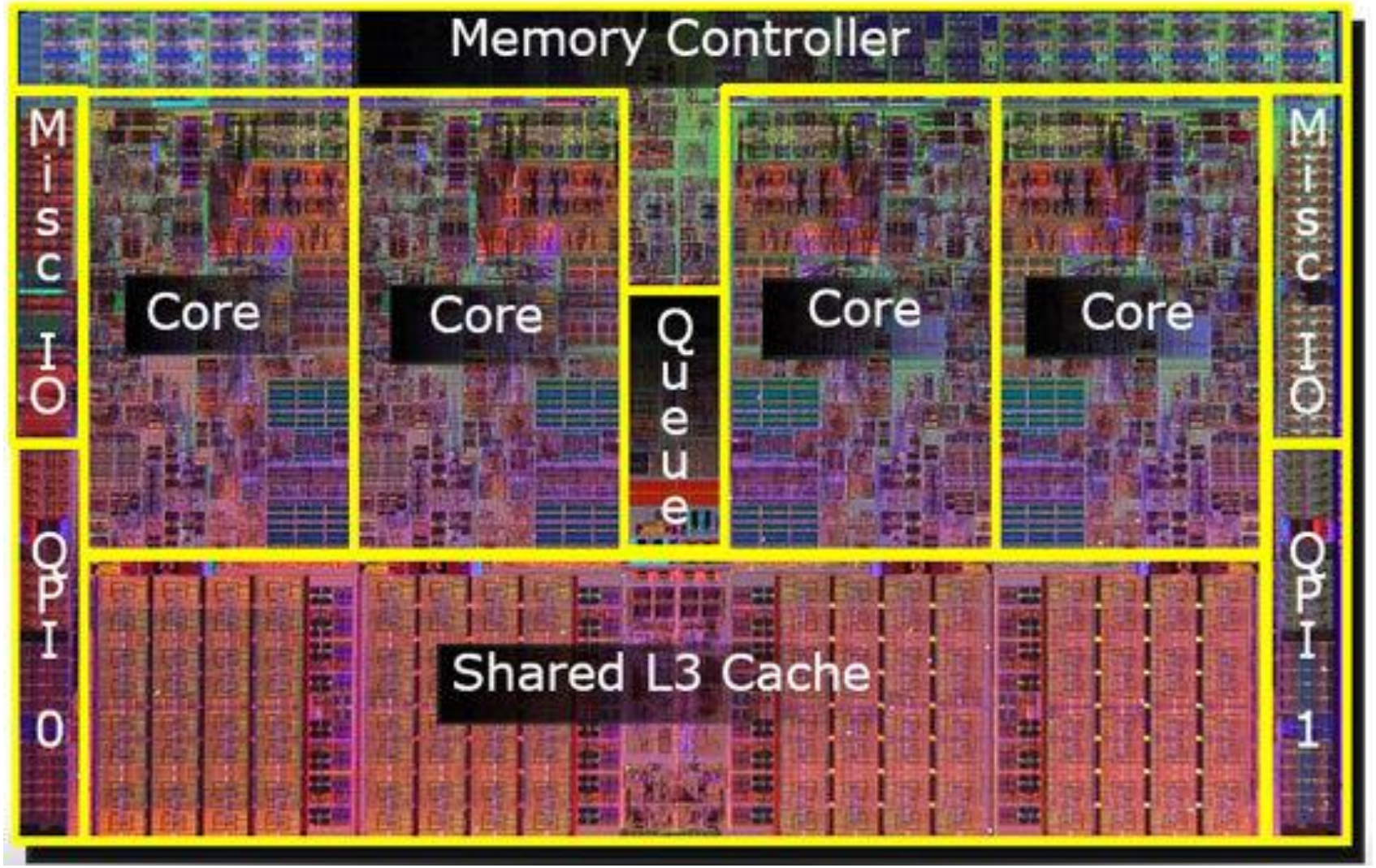
Source: M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, C. Batten (1970-2010). K. Rupp (2010-2017).

Stagnation in single-thread performance made parallel computing mainstream as transistors were invested in adding **more cores** to processors

Multi-core Processors



Real Stuff: Core i7



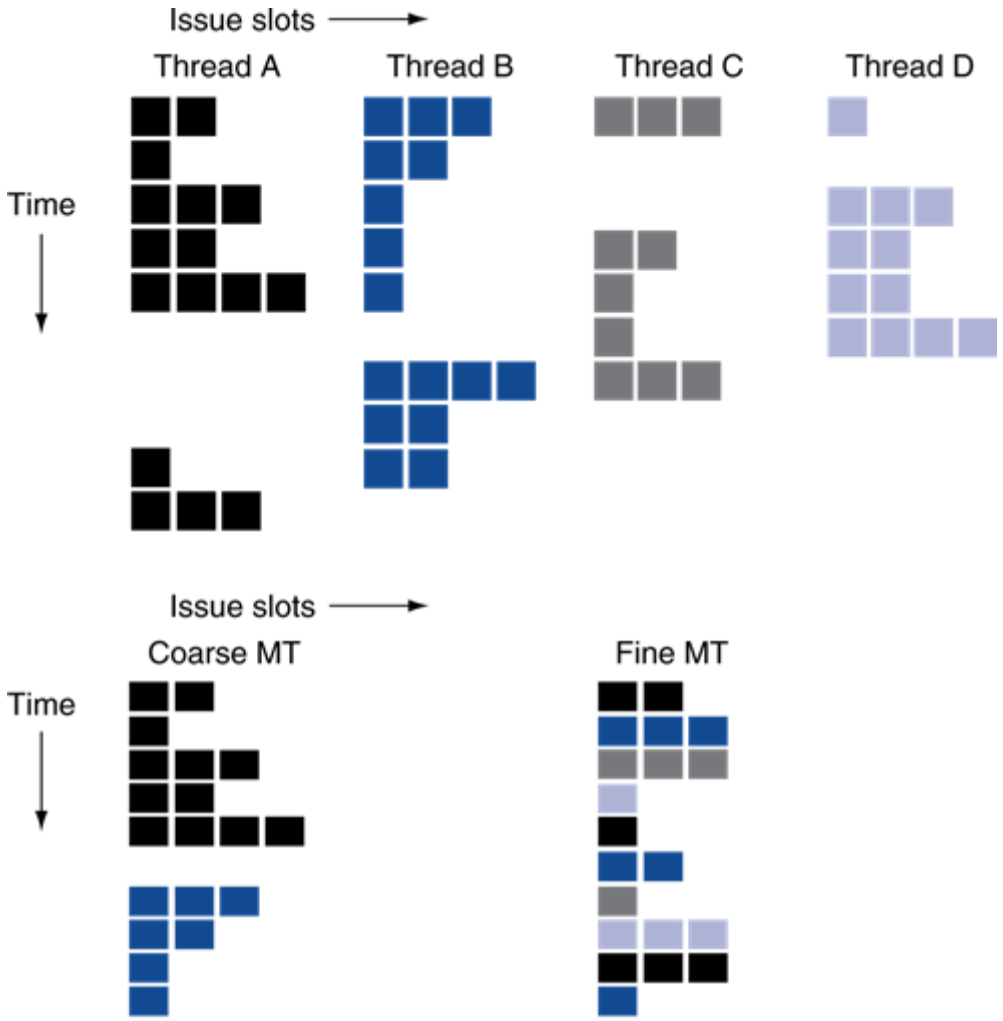
Recall: Pipeline Stalls

- Pipelines stall due to hazards
  - Exacerbated when pipelines ae made deeper and wider
- How to fill those stalls with useful work?
  - Execute instructions from other threads on the same core

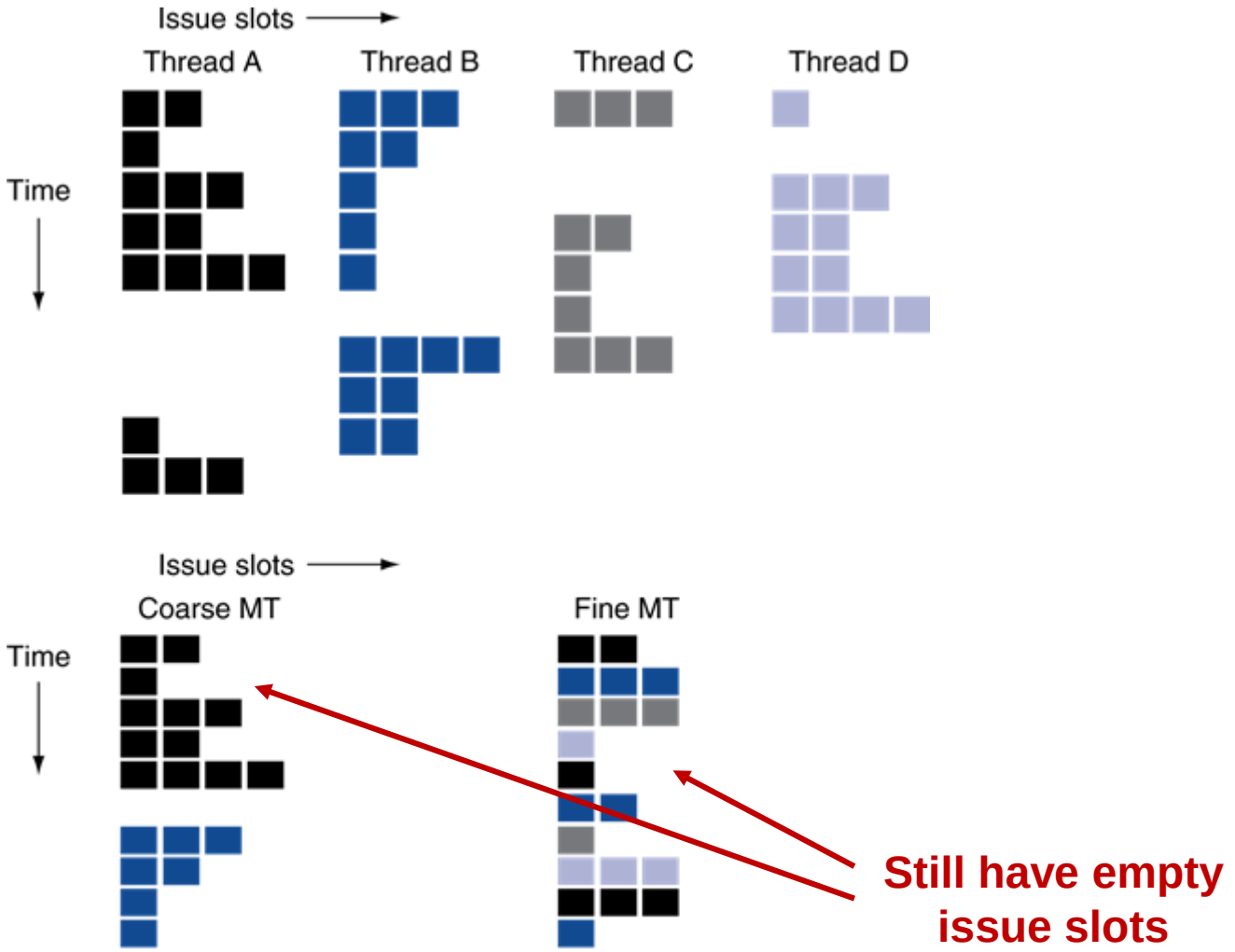
Multithreading

- **Multithreading**: executing multiple threads on the same core
  - Replicate registers, PC, etc.
  - Share functional units
- **Fine-grain multithreading**
  - Switch threads after each cycle
  - If one thread stalls, others are executed
- **Coarse-grain multithreading**
  - Only switch on long stalls (e.g., L2-cache miss)
  - Simplifies hardware, but doesn't hide short stalls (e.g., data hazards)

Multithreading Example



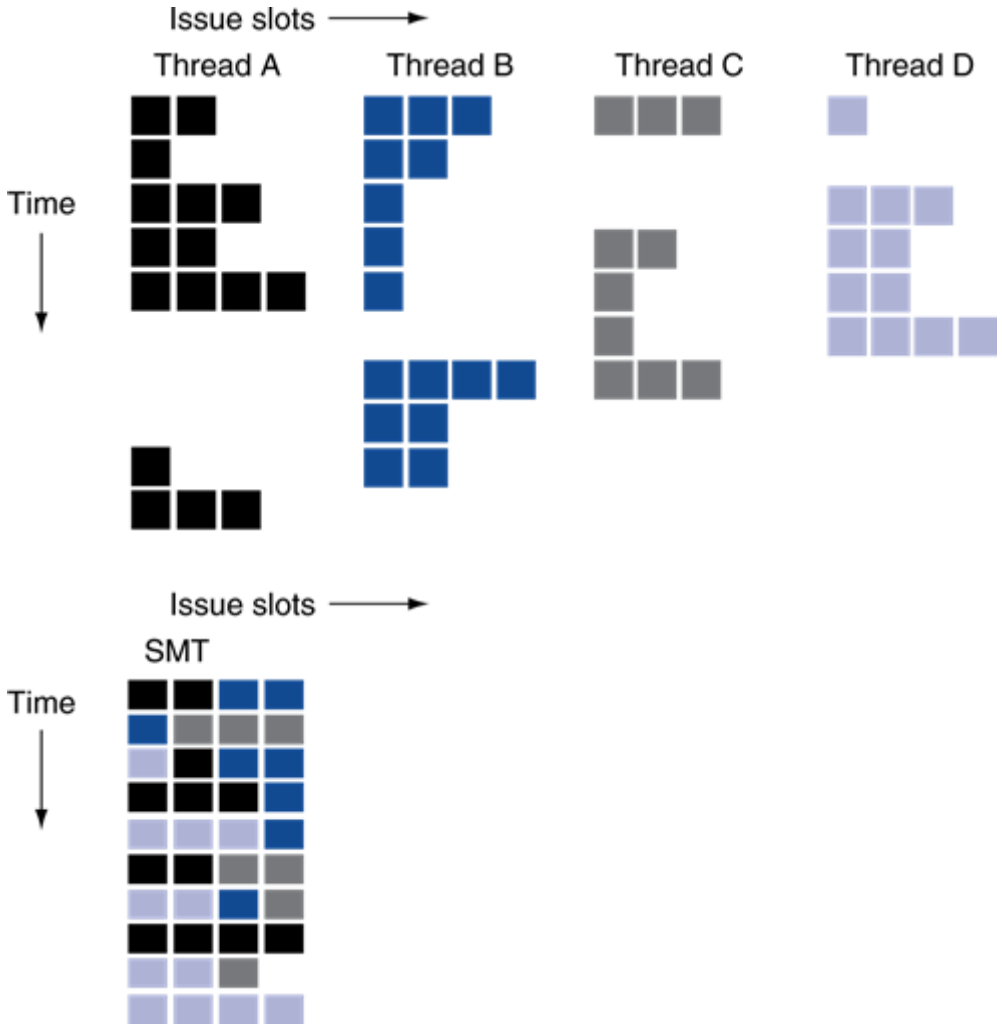
Multithreading Example



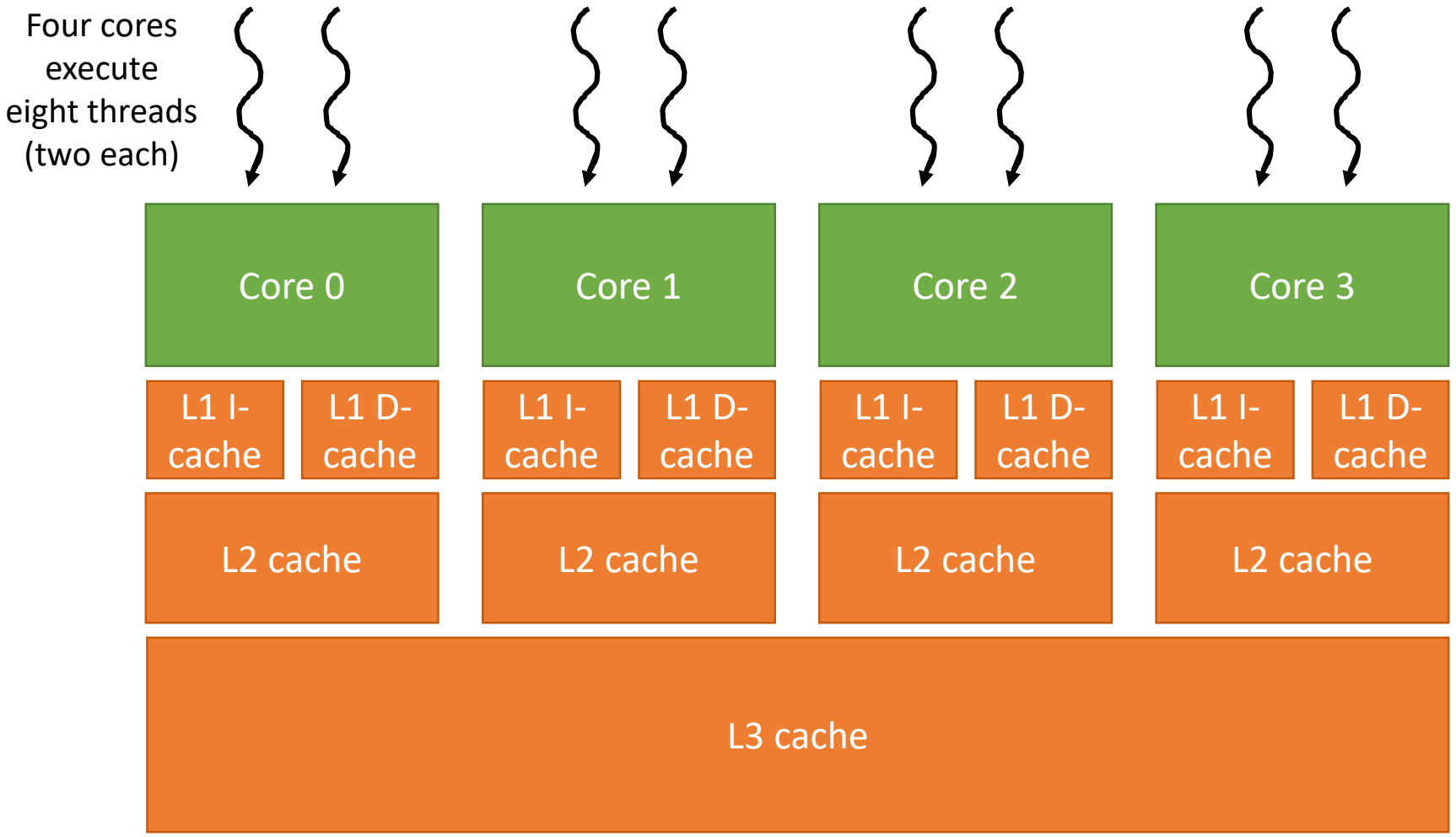
Simultaneous Multithreading

- **Simultaneous multithreading (SMT)**
  - Schedule instructions from multiple threads simultaneously
  - In a multiple-issue dynamically scheduled processor (i.e., superscalar)
- Typically: two threads per core on a modern CPU

SMT Example


Issue slots
Thread A
Thread B
Thread C
Thread D
Time
Issue slots
SMT
Time

Multi-core Processor with SMT



Instruction and Data Streams

- Flynn's taxonomy

		Data Streams	
		Single	Multiple
Instruction Streams	Single		
	Multiple		

Instruction and Data Streams

- Flynn's taxonomy

		Data Streams	
		Single	Multiple
Instruction Streams	Single	**SISD**: Sequential processor	
	Multiple		**MIMD**: Multi-core processor

Instruction and Data Streams

- Flynn's taxonomy

		Data Streams	
		Single	Multiple
Instruction Streams	Single	**SISD**: Sequential processor	**SIMD**: Vector processor
	Multiple		**MIMD**: Multi-core processor

Instruction and Data Streams

- Flynn's taxonomy

		Data Streams	
		Single	Multiple
Instruction Streams	Single	**SISD**: Sequential processor	**SIMD**: Vector processor
	Multiple	**MISD**: No examples today	**MIMD**: Multi-core processor

SIMD

- All units execute the same instruction at the same time
  - Each with a different data address
- Operate elementwise on vectors of data
  - E.g., MMX and SSE instructions in x86
    - Multiple data elements in 128-bit wide registers
- Reduced instruction control hardware
  - Amortize the cost of instruction fetch and decode over more actual computations

Graphics Processing Units

- **Graphics Processing Units** (GPUs)
  - Processors originally oriented at graphics tasks
    - Vertex/pixel processing, shading, texture mapping, rasterization
  - Design focused on processing a large number of pixels within a time constraint
    - i.e., optimized for throughput
  - Later evolved to more general purpose throughput-oriented processors

Recall: Which has better performance?

It depends on the *performance metric* we care about

Design Approaches

Latency-Oriented Design

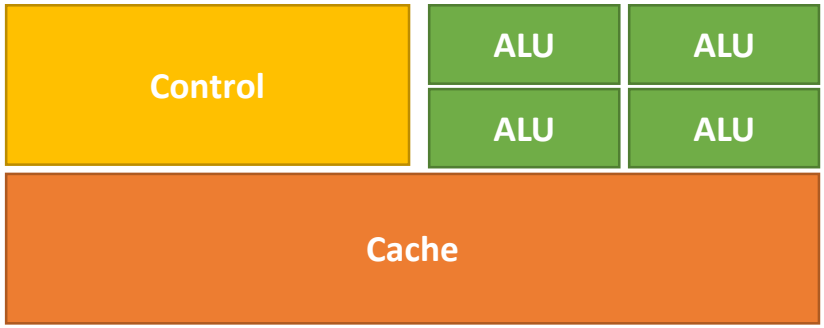
Minimize the time
it takes to perform
a single task

Throughput-Oriented Design

Maximize the number of
tasks that can be performed
in a given time frame

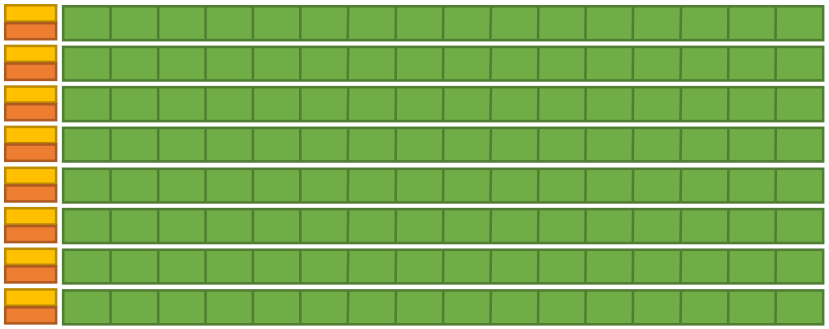
Approaches to Processor Design

CPU: Latency-Oriented Design



- A few powerful **ALUs**
  - Reduced operation latency
- Large **caches**
  - Convert long latency memory accesses to short latency cache accesses
- Sophisticated **control**
  - Branch prediction to reduce control hazards
  - Data forwarding to reduce data hazards
- Modest multithreading to hide short latency (e.g., two threads per core)

GPU: Throughput-Oriented Design

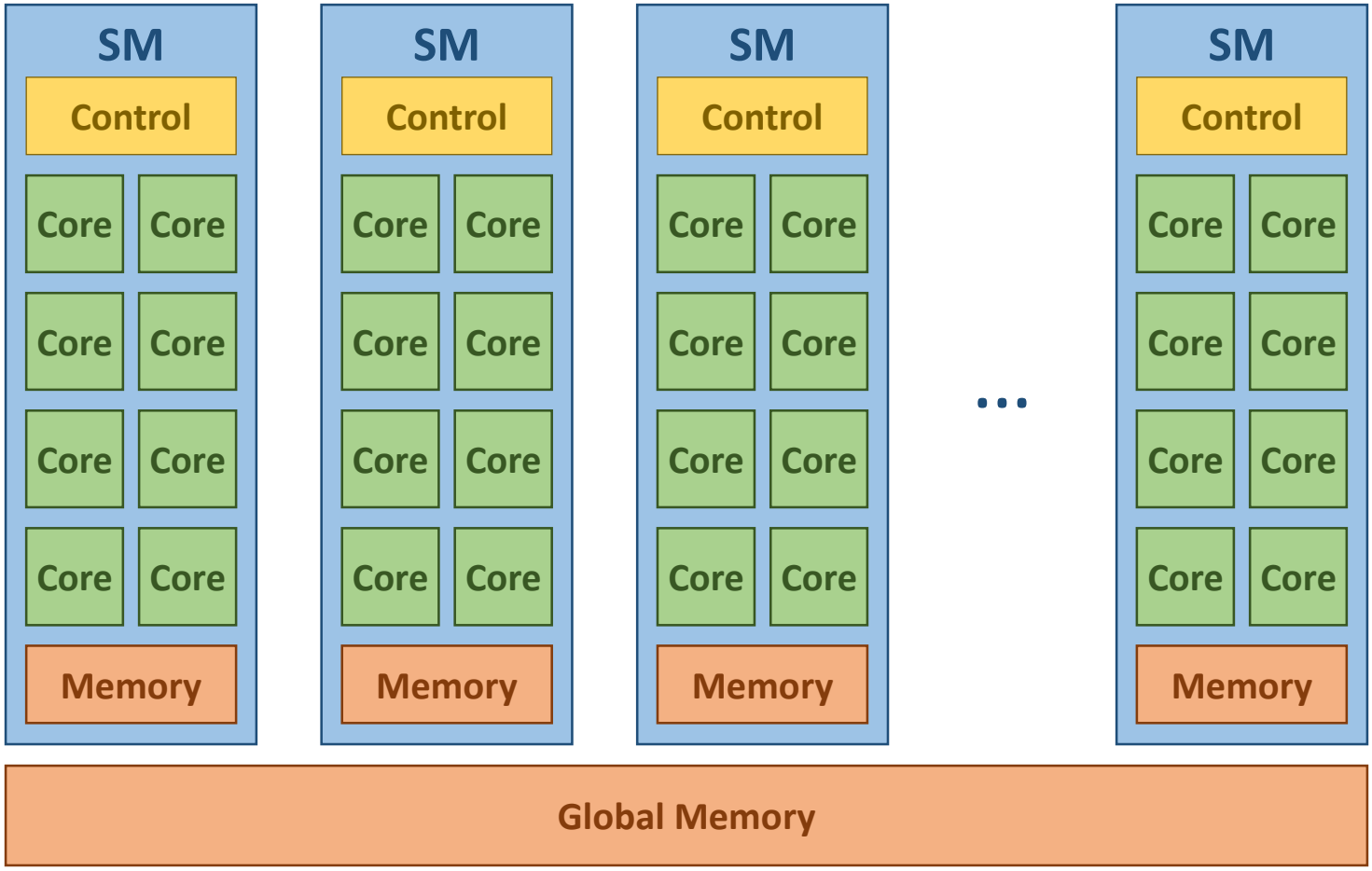


- Many small ALUs
  - Long latency, high throughput
  - Heavily pipelined for further throughput
- Small **caches**
  - More area dedicated to computation
- Simple **control**
  - More area dedicated to computation
  - SIMD execution to amortize cost of control
- Massive number of threads to hide the very high latency (e.g., 32 threads per core)

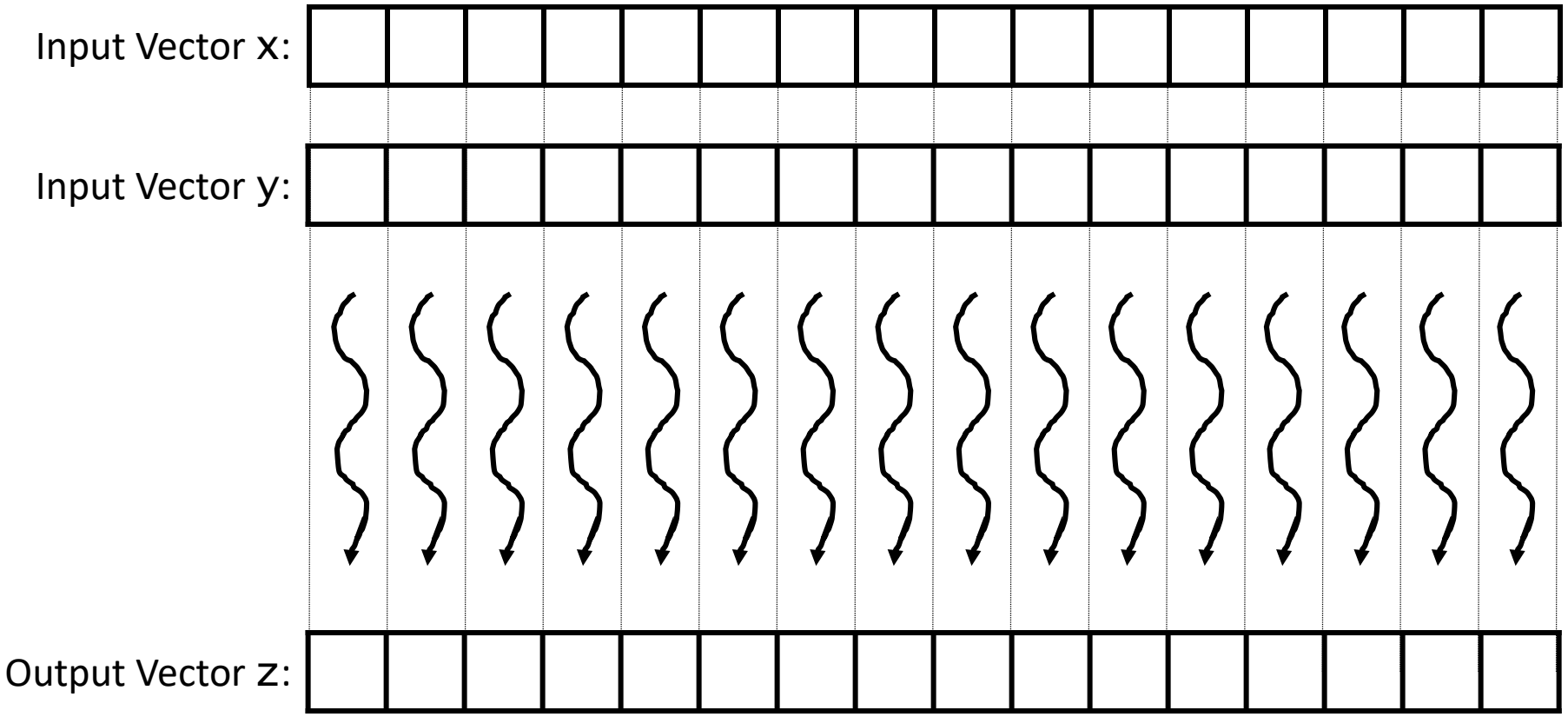
GPU Architecture

A GPU consists of multiple **Streaming Multiprocessor (SMs)**, each consisting of multiple **cores** with shared **control** and **memory**

(e.g., an NVIDIA Ampere A100 GPU has 108 SMs with 64 cores each which totals 6912 cores)

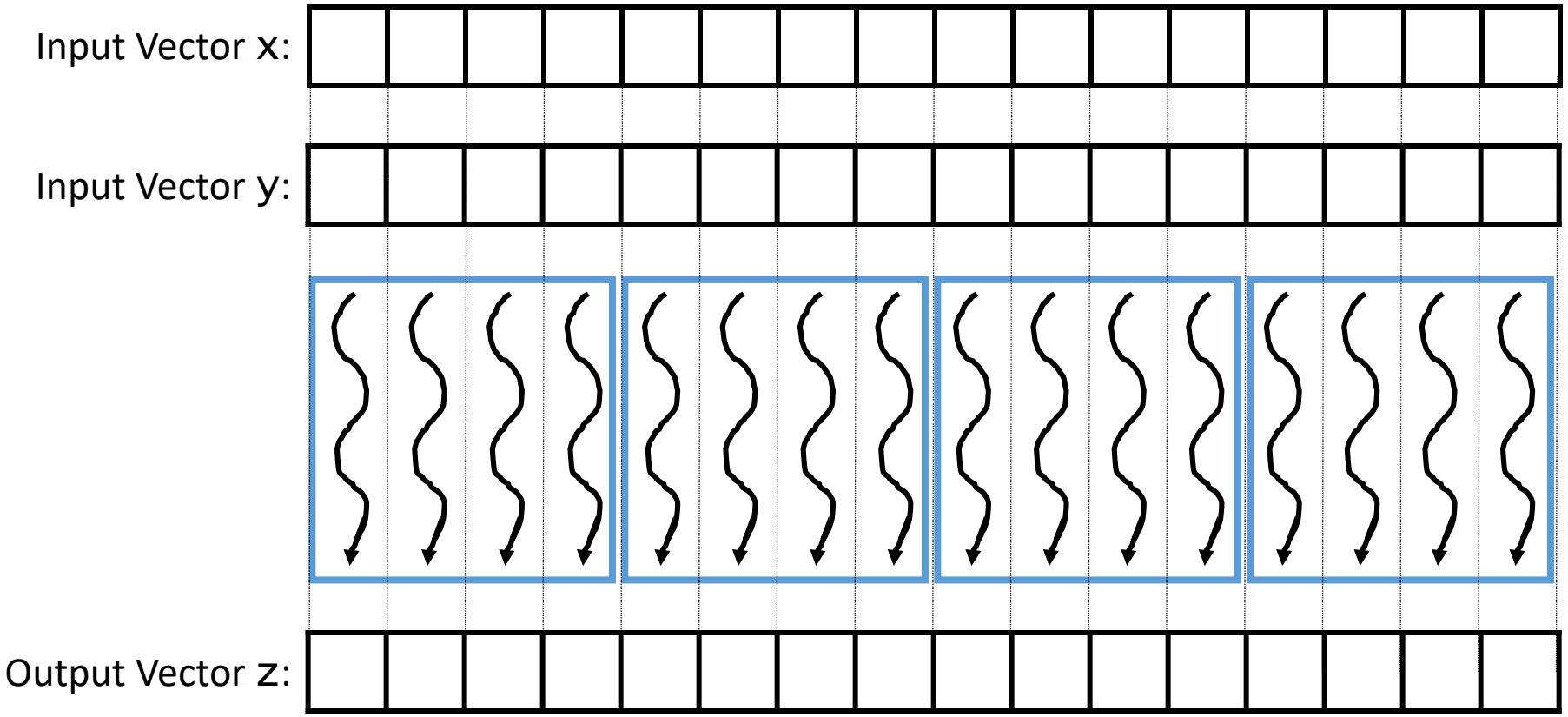


Parallel Vector Addition



To extract a massive number of threads:
assign one GPU thread per vector element

Parallel Vector Addition in CUDA




```
__global__ void vecadd_kernel(float* x, float* y, float* z, int N) {
    int i = blockDim.x*blockIdx.x + threadIdx.x;
    z[i] = x[i] + y[i];
}
```

Identify a thread's global index and use it to index the arrays

Textbook Sections

- The content in these slides corresponds to:
  - Textbook:
    - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
  - Sections:
    - 6.1-6.6